Multiple URLs for a Single Resource in ASP.NET Core Web API

Back to: ASP.NET Core Web API Tutorials

Single Resource with Multiple URLs in ASP.NET Core Web API

In this article, I will discuss **how to set up Multiple URLs for a Single Resource in the ASP.NET Core Web API** with Examples. Please read our previous article discussing working with **Route Data and Query Strings in ASP.NET Core Web API**.

In RESTful API design, routing flexibility is essential for supporting different client needs. Sometimes, different consumers (e.g., internal tools, mobile apps, or third-party clients) expect different endpoint names, even if they all serve the same resource.

Example Use Case:

- /api/Employee/GetAll → used by admin dashboard.
- /api/Employee/AllEmployees → used by mobile client.
- /api/Employee/All → used by testing or legacy systems.

ASP.NET Core provides a very **powerful and declarative routing system** that allows developers to define multiple routes for a single resource simply by decorating multiple [HttpGet], [HttpPost], etc. attributes on the same method. So, this flexibility supports:

- **Avoids Code Duplication:** Uses the same code base for multiple routes.
- **Backward Compatibility:** Older clients can still use old routes.
- **SEO-friendly or human-readable URLs:** Some APIs prefer descriptive URLs.
- **Different client preferences:** Mobile vs. web clients may follow different naming conventions.

How to Access a Single Resource with Multiple URLs?

A single resource (like all employee data) can be reached through multiple URLs without duplicating the code or creating separate methods. You achieve this by:

- Using **attribute routing**, not conventional routing.
- Decorating the same action method with **multiple route attributes**.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Each attribute defines an alternative URL pattern for accessing the same underlying logic. Let us proceed and understand how to access a single resource through multiple URLs using a real-time example with in-memory data.

Define the Employee Model

Create a class file named **Employee.cs** within the **Models** folder and then copy and paste the following code. The Employee model acts as the *data* blueprint for your API. It defines the properties (Id, Name, Gender, Department, City) that represent an employee entity. This is the core data entity that our Web API will expose to clients.

```
namespace RoutingInASPNETCoreWebAPI.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public string Department { get; set; }
        public string City { get; set; }
    }
}
```

Initialize In-Memory Data

To avoid database setup during early learning or testing, we use an **in-memory list** to simulate a data store. So, create a class file named **EmployeeData.cs** within the Models folder and then copy and paste the following code. Instead of connecting to a database, it uses a List<Employee> stored in a static class. It acts as a temporary in-memory data store for demonstration and testing.

```
using RoutingInASPNETCoreWebAPI.Models;
namespace RoutingInASPNETCoreWebAPI.Data
{
    public static class EmployeeData
    {
        public static List<Employee> Employees = new List<Employee>
        {
            new Employee { Id = 1, Name = "Alice Johnson", Gender = "Female",
Department = "HR", City = "New York" },
            new Employee { Id = 2, Name = "Bob Smith", Gender = "Male",
Department = "IT", City = "Los Angeles" },
            new Employee { Id = 3, Name = "Charlie Davis", Gender = "Male",
Department = "Finance", City = "Chicago" }
            // Add more employees as needed
        };
    }
}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Implementing Multiple Routes for a Single Resource in ASP.NET Core Web API

Now, we will create an EmployeeController that exposes endpoints to retrieve employee data. We will configure multiple routes for accessing the same resource. Suppose we have the following resource available in our Employee Controller.

```
public ActionResult<IEnumerable<Employee>> GetAllEmployees()
{
    var employees = EmployeeData.Employees;
    return Ok(employees);
}
```

Now, we want to access the above resource with three URLs:

- **api/Employee/All**
- **api/Employee/AllEmployees**
- **api/Employee/GetAll**

How can we do this? If this is your requirement, you need to decorate the **GetAllEmployees** action method with three different **HttpGet** Attributes. So, please modify the EmployeeController class as shown in the code below.

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Models;
using RoutingInASPNETCoreWebAPI.Data;

namespace EmployeeApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // Action Method with Multiple Routes
        [HttpGet("All")]
        [HttpGet("AllEmployees")]
        [HttpGet("GetAll")]
        public ActionResult<IEnumerable<Employee>> GetAllEmployees()
        {
            var employees = EmployeeData.Employees;
            return Ok(employees);
        }
    }
}
```

Code Explanation

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The GetAllEmployees action is decorated with 3 [HttpGet] attributes, each specifying a different route:

- **api/Employee/All**
- **api/Employee/AllEmployees**
- **api/Employee/GetAll**

This configuration allows clients to access the same resource using any of these URLs. Now, run the application and access the above resource using the different URLs, and you should get the same result as shown in the image below.

Using a web browser or Postman, accessing any of the URLs should display the same list of employees, confirming that the GetAllEmployees method is accessible via multiple routes. As you can observe, each URL is unique. So, as long as the URLs are unique, you can access a particular resource using different URLs.

What Happens if Two Action Methods Use the Same Route in ASP.NET Core Web API?

Assigning the same route to multiple action methods can lead to conflicts, and routing becomes ambiguous. ASP.NET Core enforces unique routes for each action to prevent ambiguity. Let us prove this with an example. Please modify the Employee Controller as follows. Here, you can see we have two action methods in the EmployeeController with the same route, i.e., api/Employee/All:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace EmployeeApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // First Action Method
        [HttpGet("All")]
        public ActionResult<IEnumerable<Employee>> GetAllEmployees()
        {
            var employees = EmployeeData.Employees;
            return Ok(employees);
        }

        // Second Action Method with Same Route
        [HttpGet("All")]
        public ActionResult<IEnumerable<Employee>> GetEmployees()
        {
            var employees = EmployeeData.Employees;
            return Ok(employees);
        }
    }
}
```

Now, within the IDE, you might be getting a warning as shown in the image below:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

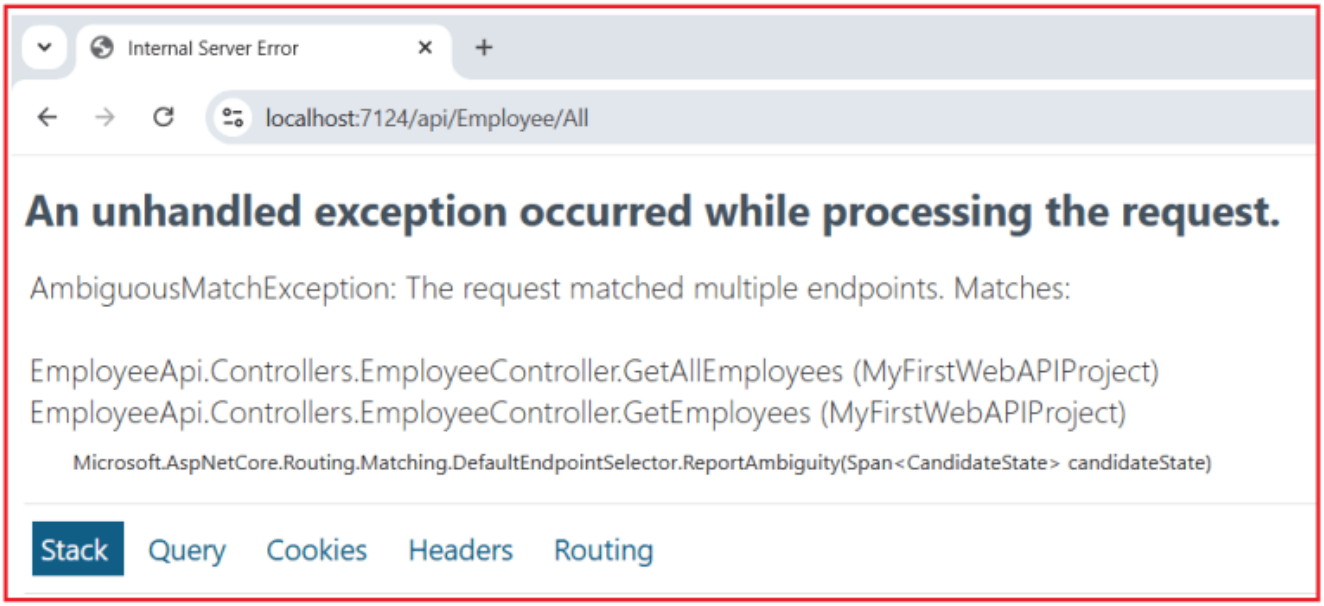
```
[ApiController]
[Route("api/[controller]")]
0 references
public class EmployeeController : ControllerBase
{
    // First Action Method
    [HttpGet("All")]
    0 references
    public Actio
    {
        var empl
        return O
    }

    // Second Action Method with Same Route
    [HttpGet("All")]
    0 references
    public ActionResult<IEnumerable<Employee>> GetEmployees()
    {
        var employees = EmployeeData.Employees;
        return Ok(employees);
    }
}
```

class System.String
Represents text as a sequence of UTF-16 code units.

ASP0023: Route 'All' conflicts with another action route. An HTTP request that matches multiple routes results in an ambiguous match error. Fix the conflict by changing the route's pattern, HTTP method, or route constraints.

With the above changes, attempting to access the **api/Employee/All** endpoint will result in an Internal Server Error with an AmbiguousMatchException, as shown in the image below.



As you can see in the above image, it is throwing an Internal Server Error saying **AmbiguousMatchException**: The request matched multiple endpoints. The reason for this error is that the ASP.NET Core routing system cannot determine which action method to invoke when multiple actions share the same route. This ambiguity leads to a runtime exception.

Resolving Route Conflicts in ASP.NET Core Web API

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

To resolve such route conflicts, ensure that each action method has a unique route. Now, change the Route Attribute of both the resources shown in the code below to give distinct URLs to each route.

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace EmployeeApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        // First Action Method with Unique Routes
        [HttpGet("GetAll")]
        [HttpGet("AllEmployees")]
        public ActionResult<IEnumerable<Employee>> GetAllEmployees()
        {
            var employees = EmployeeData.Employees;
            return Ok(employees);
        }

        // Second Action Method with a Different Route
        [HttpGet("All")]
        public ActionResult<IEnumerable<Employee>> GetEmployees()
        {
            var employees = EmployeeData.Employees;
            return Ok(employees);
        }
    }
}
```

With the above changes in place, you can now access the **GetAllEmployees** resource using two URLs, i.e., **api/Employee/GetAll and api/Employee/AllEmployees**. On the other hand, you can access the **GetEmployees** resource using the URL **api/Employee/All**.

Real-time Analogy:

Think of your **ASP.NET Core Web API** like a **customer service call center**:

- Each **Action Method** is like a **Customer Service Representative (CSR)** trained to handle a specific type of query.
- Each **URL (Route)** is like a **Phone Number** or **Extension** that routes the caller to the right representative.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

You can have **Multiple Phone Numbers (URLs)** reach the same representative if you want to serve customers coming from different regions or departments.

But you **cannot assign the same Phone Number to Two Different Representatives**, or the call router (Routing Engine) will get confused and not know who should answer, resulting in a failed connection (runtime error).

Real-time Example of Multiple Routes of an Action Method: Endpoint Naming Changes

There may be times when you realize an endpoint name is not informative or needs to be rebranded. However, if you change it immediately, existing clients will break. Keeping both the old and new endpoint names simultaneously allows for a smooth transition.

For example, we previously named our endpoint as /api/old-employees. After some organizational changes, we decided that /api/staff is a clearer name. We want to keep /api/old-employees around until all our clients switch over.

```
using Microsoft.AspNetCore.Mvc;
using RoutingInASPNETCoreWebAPI.Data;
using RoutingInASPNETCoreWebAPI.Models;

namespace EmployeeApi.Controllers
{
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        // Multiple route attributes to support old and new naming
        [Route("api/old-employees")]
        [Route("api/staff")]
        [HttpGet]
        public ActionResult<IEnumerable<Employee>> GetAllEmployees()
        {
            var employees = EmployeeData.Employees;
            return Ok(employees);
        }
    }
}
```

Here,

- api/old-employees → Old endpoint, still functional for backward compatibility.
- api/staff → New, more descriptive endpoint.

Clients can use either URL without changing the underlying logic. Over time, you can phase out the old endpoint if usage drops to zero.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

By using attribute routing in ASP.NET Core Web API, we can configure multiple URLs for a single resource. However, remember that each URL must be unique for each resource to avoid route conflicts and errors.

In the next article, I will discuss **Token Replacement in ASP.NET Core Web API Attribute Routing** with Examples. In this article, I explain **how to set up Multiple URLs for a Single Resource in an ASP.NET Core Web API** Application with Examples. I hope you enjoy using multiple URLs for a single resource in the ASP.NET Core Web API article.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent Do Not Sell My Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF